Experiment 3

Creation of a Simple Network Topology Using Open-Source Network Virtualization Tools

Name: Mantu Kushali | Adamas University | Cloud Computing Lab | 23 September 2026

Objective

To create and configure a simple network topology using an open-source network virtualization tool and to verify communication between the virtual network nodes.

Requirements

Component	Details
Operating system	Ubuntu Linux Virtual Machine (hostname: ubuntu-lab1)
Virtualization	Oracle VirtualBox
Network emulator	Mininet 2.3.0 with Open vSwitch (OVS Bridge)
Tools used	`ping` and basic Linux networking commands

Theory

Mininet is an open-source network emulator that builds a complete virtual network - hosts, switches, links and controllers - on a single Linux machine. Each virtual host runs in its own network namespace and behaves like a real machine, while switches are provided by Open vSwitch. The default topology, a single switch connected to two hosts, is enough to test connectivity with standard tools such as `ping`.

Procedure

#	Step	Command
1	Install Mininet on the Ubuntu VM and confirm the version.	`sudo apt install mininet` `mn --version`
2	Start the default topology (1 switch, 2 hosts).	`sudo mn`

3	Inspect the topology from the Mininet CLI.	`nodes`
		`net`
		`dump`

4	Test connectivity between all hosts.	`pingall`
5	Ping from h1 to h2 with four packets.	`h1 ping -c 4 h2`
6	Verify link status.	`links`
7	Stop the network and clean up leftover processes and interfaces.	`exit` `sudo mn -c`

Output and Observations

1. Installation, topology creation and inspection

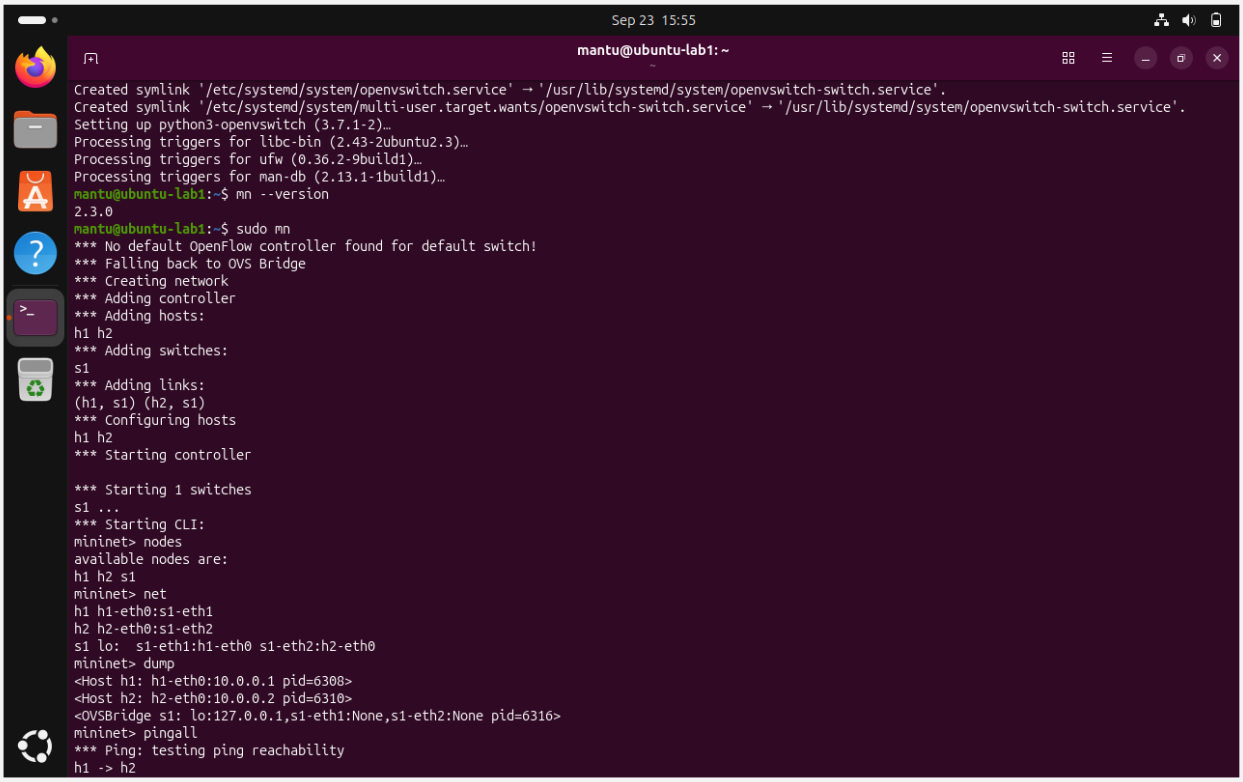


Fig. 1: Mininet 2.3.0 installed; `sudo mn` *creates the topology;* `nodes`, `net`, `dump` *and* `pingall` *executed.*

Topology summary (from `net` and `dump`)

Node	Interface	IP address	Connected to
h1 (host)	h1-eth0	10.0.0.1	s1-eth1
h2 (host)	h2-eth0	10.0.0.2	s1-eth2
s1 (OVS bridge)	s1-eth1, s1-eth2	-	h1-eth0, h2-eth0

2. Connectivity test and stopping the network

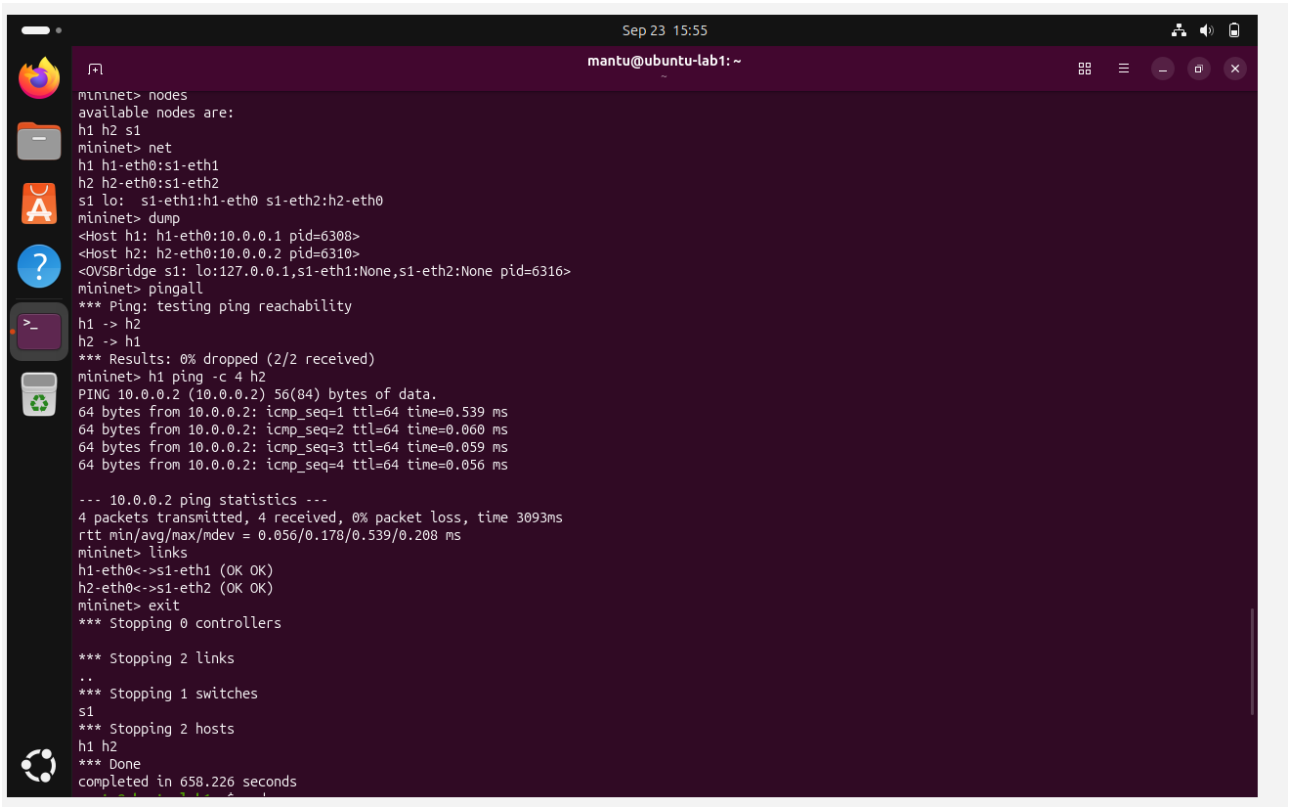


Fig. 2: `pingall` *shows 0% dropped;* `h1 ping -c 4 h2` *succeeds;* `links` *reports both links OK;* `exit` *stops the network.*

Ping results

Measurement	Result
pingall	0% dropped (2/2 received)
Packets transmitted / received	4 / 4 (0% packet loss)
RTT min / avg / max / mdev	0.056 / 0.178 / 0.539 / 0.208 ms
TTL	64
Links	h1-eth0 <-> s1-eth1 (OK OK); h2-eth0 <-> s1-eth2 (OK OK)

3. Stopping the network and cleanup

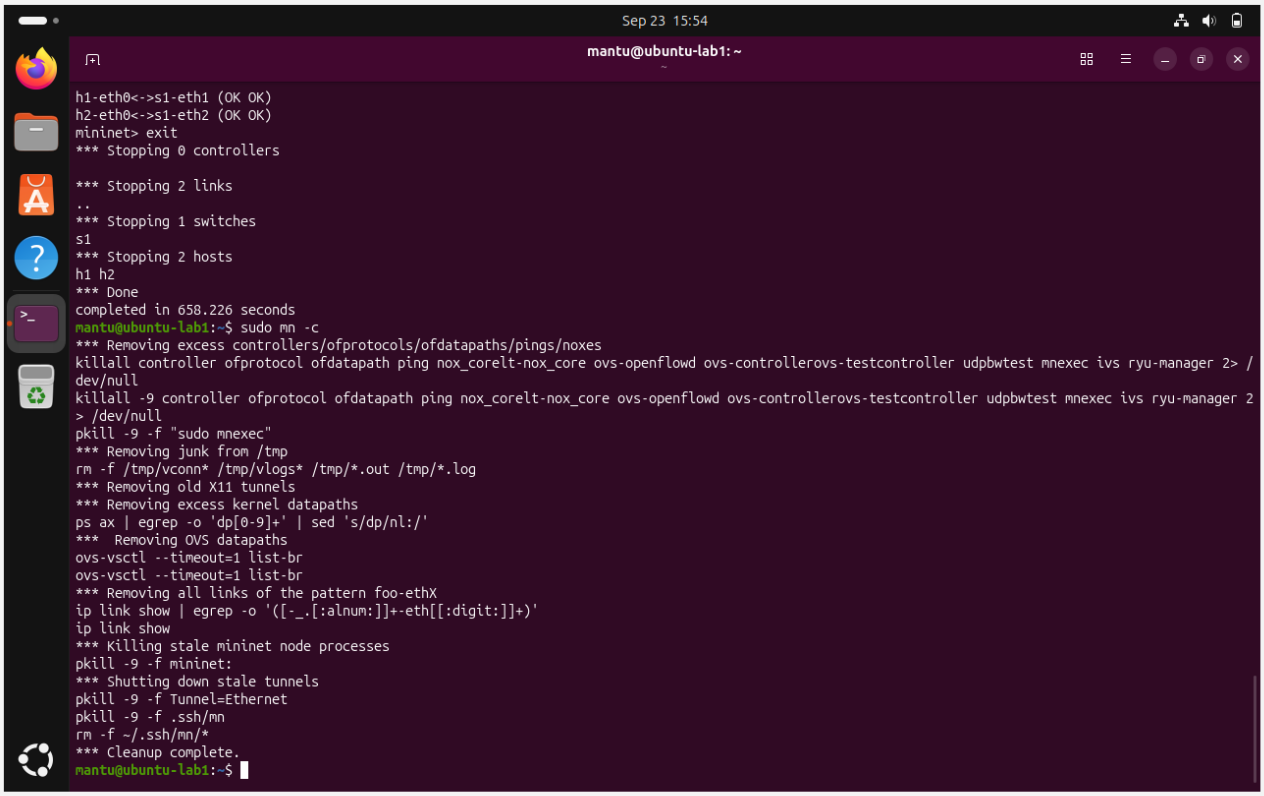


Fig. 3: After `exit`, Mininet stops 0 controllers, 2 links, 1 switch and 2 hosts. `sudo mn -c` then removes stale processes, OVS datapaths and links.

Exiting the CLI shuts down the emulated hosts, switch and links in order. Running `sudo mn -c` afterwards clears any leftover Mininet state, so the next experiment starts from a clean environment.

Result

A simple network topology with one Open vSwitch bridge (s1) and two hosts (h1, h2) was created in Mininet 2.3.0 on an Ubuntu VM running in VirtualBox. Connectivity was verified with `pingall` and `ping`, with no packet loss. The network was then stopped and cleaned up successfully.

Conclusion

Mininet lets a complete virtual network be built and tested on a single machine without physical hardware. The experiment demonstrated topology creation, node and link inspection, connectivity testing with `ping`, and proper shutdown and cleanup of the emulated network.